EAST WEST UNIVERSITY

Course Code: CSE423

Course Title: Software Architecture

Section: 01

Semester: Spring26

Group No: 01

Report

Project Name: Hospital Management System

Submitted To:

Ahmed Adnan

Lecturer

Department of Computer Science and Engineering

Submitted By:

Name:	ID:
Sababa Hoque Saba	2022-3-60-060
Afia Ayub Samirah	2022-3-60-012
Kha. Mo. Syeed Asif	2022-3-60-030
Md. Omor Faruq Shikdere	2022-1-60-335

Date of Submission: 07-04-2026

Design Pattern

Creational Pattern(FACTORY PATTERN)

Before:

```
Doctor doctor = new Doctor(name, mobile, gender, department, joiningDate, salary, room, bmdc);
```

We replace this.

After:

Create DoctorFactory.java

DoctorFactory.java

```
public class DoctorFactory {


  public Doctor createDoctor(

        String name,

        String mobile,

        String gender,

        String department,

        String joiningDate,

        double salary,

        String roomNo,

        String bmdcReg

  ) {
```

```
        return new Doctor(
                name,
                mobile,
                gender,
                department,
                joiningDate,
                salary,
                roomNo,
                bmdcReg
        );
    }
}
```

Use in AddDoctorUI

Before **AddDoctorUI.java**

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class AddDoctorUI extends JFrame {

    private IDoctorService doctorService;
```

```java
private JTextField nameTF, mobileTF, joiningTF, salaryTF, roomTF, bmdcTF;

private JRadioButton maleRB, femaleRB, otherRB;

private JComboBox<String> deptCB;

private JButton addButton, backButton;

private ButtonGroup genderGroup;


public AddDoctorUI(IDoctorService doctorService) {

    super("Add Doctor Form");

    this.doctorService = doctorService;


    setSize(900, 600);

    setLocationRelativeTo(null);

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);


    initUI();

}


private void initUI() {

    JPanel panel = new JPanel();

    panel.setLayout(null);

    panel.setBackground(new Color(143,207,225));
```

```
Font labelFont = new Font("Cascadia Code SemiBold", Font.PLAIN, 18);


// Labels & TextFields

JLabel titleLabel = new JLabel("Add Doctor Form");

titleLabel.setBounds(130, 20, 300, 40);

titleLabel.setFont(new Font("Biome", Font.BOLD, 30));

panel.add(titleLabel);


JLabel nameLabel = new JLabel("Doctor Name:");

nameLabel.setBounds(75, 80, 120, 25);

nameLabel.setFont(labelFont);

panel.add(nameLabel);


nameTF = new JTextField();

nameTF.setBounds(200, 80, 150, 25);

panel.add(nameTF);


JLabel mobileLabel = new JLabel("Mobile Number:");

mobileLabel.setBounds(75, 120, 120, 25);

mobileLabel.setFont(labelFont);

panel.add(mobileLabel);


mobileTF = new JTextField();
```

```
mobileTF.setBounds(200, 120, 150, 25);

panel.add(mobileTF);


JLabel genderLabel = new JLabel("Gender:");

genderLabel.setBounds(75, 160, 100, 25);

genderLabel.setFont(labelFont);

panel.add(genderLabel);


maleRB = new JRadioButton("Male");

maleRB.setBounds(150, 160, 80, 25);

maleRB.setBackground(panel.getBackground());

femaleRB = new JRadioButton("Female");

femaleRB.setBounds(230, 160, 80, 25);

femaleRB.setBackground(panel.getBackground());

otherRB = new JRadioButton("Other");

otherRB.setBounds(310, 160, 80, 25);

otherRB.setBackground(panel.getBackground());


genderGroup = new ButtonGroup();

genderGroup.add(maleRB);

genderGroup.add(femaleRB);

genderGroup.add(otherRB);
```

```
        panel.add(maleRB);

        panel.add(femaleRB);

        panel.add(otherRB);


        JLabel deptLabel = new JLabel("Department:");

        deptLabel.setBounds(75, 200, 120, 25);

        deptLabel.setFont(labelFont);

        panel.add(deptLabel);


        String[] departments = {"CARDIOLOGY", "NEUROSURGERY", "ONCOLOGIST",
"ORTHOPEDICS"};

        deptCB = new JComboBox<>(departments);

        deptCB.setBounds(200, 200, 150, 25);

        panel.add(deptCB);


        JLabel joiningLabel = new JLabel("Joining Date:");

        joiningLabel.setBounds(75, 240, 120, 25);

        joiningLabel.setFont(labelFont);

        panel.add(joiningLabel);


        joiningTF = new JTextField();

        joiningTF.setBounds(200, 240, 150, 25);

        panel.add(joiningTF);
```

```
JLabel salaryLabel = new JLabel("Salary:");
salaryLabel.setBounds(75, 280, 120, 25);
salaryLabel.setFont(labelFont);
panel.add(salaryLabel);

salaryTF = new JTextField();
salaryTF.setBounds(200, 280, 150, 25);
panel.add(salaryTF);

JLabel roomLabel = new JLabel("Room No:");
roomLabel.setBounds(75, 320, 120, 25);
roomLabel.setFont(labelFont);
panel.add(roomLabel);

roomTF = new JTextField();
roomTF.setBounds(200, 320, 150, 25);
panel.add(roomTF);

JLabel bmdcLabel = new JLabel("BM&DC REG:");
bmdcLabel.setBounds(75, 360, 120, 25);
bmdcLabel.setFont(labelFont);
panel.add(bmdcLabel);
```

```
    bmdcTF = new JTextField();
    bmdcTF.setBounds(200, 360, 150, 25);
    panel.add(bmdcTF);

    // Buttons
    addButton = new JButton("ADD");
    addButton.setBounds(500, 470, 100, 40);
    addButton.setBackground(Color.GREEN);
    addButton.setForeground(Color.WHITE);
    addButton.addActionListener(e -> onAdd());
    panel.add(addButton);

    backButton = new JButton("BACK");
    backButton.setBounds(620, 470, 100, 40);
    backButton.setBackground(Color.RED);
    backButton.setForeground(Color.WHITE);
    backButton.addActionListener(e -> onBack());
    panel.add(backButton);

    this.add(panel);
}
```

```
  private void onAdd() {

      String name = nameTF.getText();

      String mobile = mobileTF.getText();

      String gender = maleRB.isSelected() ? "Male" :

                  femaleRB.isSelected() ? "Female" : "Other";

      String department = (String) deptCB.getSelectedItem();

      String joiningDate = joiningTF.getText();

      double salary = Double.parseDouble(salaryTF.getText());

      String room = roomTF.getText();

      String bmdc = bmdcTF.getText();



              Doctor doctor = new Doctor(name, mobile, gender, department, joiningDate, salary, room,
bmdc);

      doctorService.addDoctor(doctor);



      JOptionPane.showMessageDialog(this, "Doctor added successfully!");

      clearFields();

  }



  private void onBack() {

      this.setVisible(false);

      DrMenu dm = new DrMenu(); // Navigation to main menu

      dm.setVisible(true);
```

```
    }

    private void clearFields() {
        nameTF.setText("");
        mobileTF.setText("");
        genderGroup.clearSelection();
        deptCB.setSelectedIndex(0);
        joiningTF.setText("");
        salaryTF.setText("");
        roomTF.setText("");
        bmdcTF.setText("");
    }
}
```

After **AddDoctorUI.java**

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class AddDoctorUI extends JFrame {

    private IDoctorService doctorService;
```

```java
private JTextField nameTF, mobileTF, joiningTF, salaryTF, roomTF, bmdcTF;

private JRadioButton maleRB, femaleRB, otherRB;

private JComboBox<String> deptCB;

private JButton addButton, backButton;

private ButtonGroup genderGroup;


public AddDoctorUI(IDoctorService doctorService) {

    super("Add Doctor Form");

    this.doctorService = doctorService;


    setSize(900, 600);

    setLocationRelativeTo(null);

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);


    initUI();

}


private void initUI() {

    JPanel panel = new JPanel();

    panel.setLayout(null);

    panel.setBackground(new Color(143,207,225));
```

```
Font labelFont = new Font("Cascadia Code SemiBold", Font.PLAIN, 18);


// Labels & TextFields

JLabel titleLabel = new JLabel("Add Doctor Form");

titleLabel.setBounds(130, 20, 300, 40);

titleLabel.setFont(new Font("Biome", Font.BOLD, 30));

panel.add(titleLabel);


JLabel nameLabel = new JLabel("Doctor Name:");

nameLabel.setBounds(75, 80, 120, 25);

nameLabel.setFont(labelFont);

panel.add(nameLabel);


nameTF = new JTextField();

nameTF.setBounds(200, 80, 150, 25);

panel.add(nameTF);


JLabel mobileLabel = new JLabel("Mobile Number:");

mobileLabel.setBounds(75, 120, 120, 25);

mobileLabel.setFont(labelFont);

panel.add(mobileLabel);


mobileTF = new JTextField();
```

```
mobileTF.setBounds(200, 120, 150, 25);

panel.add(mobileTF);


JLabel genderLabel = new JLabel("Gender:");

genderLabel.setBounds(75, 160, 100, 25);

genderLabel.setFont(labelFont);

panel.add(genderLabel);


maleRB = new JRadioButton("Male");

maleRB.setBounds(150, 160, 80, 25);

maleRB.setBackground(panel.getBackground());

femaleRB = new JRadioButton("Female");

femaleRB.setBounds(230, 160, 80, 25);

femaleRB.setBackground(panel.getBackground());

otherRB = new JRadioButton("Other");

otherRB.setBounds(310, 160, 80, 25);

otherRB.setBackground(panel.getBackground());


genderGroup = new ButtonGroup();

genderGroup.add(maleRB);

genderGroup.add(femaleRB);

genderGroup.add(otherRB);
```

```
        panel.add(maleRB);

        panel.add(femaleRB);

        panel.add(otherRB);


        JLabel deptLabel = new JLabel("Department:");

        deptLabel.setBounds(75, 200, 120, 25);

        deptLabel.setFont(labelFont);

        panel.add(deptLabel);


        String[] departments = {"CARDIOLOGY", "NEUROSURGERY", "ONCOLOGIST",
"ORTHOPEDICS"};

        deptCB = new JComboBox<>(departments);

        deptCB.setBounds(200, 200, 150, 25);

        panel.add(deptCB);


        JLabel joiningLabel = new JLabel("Joining Date:");

        joiningLabel.setBounds(75, 240, 120, 25);

        joiningLabel.setFont(labelFont);

        panel.add(joiningLabel);


        joiningTF = new JTextField();

        joiningTF.setBounds(200, 240, 150, 25);

        panel.add(joiningTF);
```

```
JLabel salaryLabel = new JLabel("Salary:");
salaryLabel.setBounds(75, 280, 120, 25);
salaryLabel.setFont(labelFont);
panel.add(salaryLabel);

salaryTF = new JTextField();
salaryTF.setBounds(200, 280, 150, 25);
panel.add(salaryTF);

JLabel roomLabel = new JLabel("Room No:");
roomLabel.setBounds(75, 320, 120, 25);
roomLabel.setFont(labelFont);
panel.add(roomLabel);

roomTF = new JTextField();
roomTF.setBounds(200, 320, 150, 25);
panel.add(roomTF);

JLabel bmdcLabel = new JLabel("BM&DC REG:");
bmdcLabel.setBounds(75, 360, 120, 25);
bmdcLabel.setFont(labelFont);
panel.add(bmdcLabel);
```

```
    bmdcTF = new JTextField();
    bmdcTF.setBounds(200, 360, 150, 25);
    panel.add(bmdcTF);

    // Buttons
    addButton = new JButton("ADD");
    addButton.setBounds(500, 470, 100, 40);
    addButton.setBackground(Color.GREEN);
    addButton.setForeground(Color.WHITE);
    addButton.addActionListener(e -> onAdd());
    panel.add(addButton);

    backButton = new JButton("BACK");
    backButton.setBounds(620, 470, 100, 40);
    backButton.setBackground(Color.RED);
    backButton.setForeground(Color.WHITE);
    backButton.addActionListener(e -> onBack());
    panel.add(backButton);

    this.add(panel);
}
```

```
private void onAdd() {

    String name = nameTF.getText();

    String mobile = mobileTF.getText();

    String gender = maleRB.isSelected() ? "Male" :

                femaleRB.isSelected() ? "Female" : "Other";

    String department = (String) deptCB.getSelectedItem();

    String joiningDate = joiningTF.getText();

    double salary = Double.parseDouble(salaryTF.getText());

    String room = roomTF.getText();

    String bmdc = bmdcTF.getText();


        DoctorFactory factory = new DoctorFactory();


    Doctor doctor = factory.createDoctor(

        name, mobile, gender, department,

        joiningDate, salary, room, bmdc

    );

    doctorService.addDoctor(doctor);


    JOptionPane.showMessageDialog(this, "Doctor added successfully!");

    clearFields();

}
```

```java
    private void onBack() {
        this.setVisible(false);
        DrMenu dm = new DrMenu(); // Navigation to main menu
        dm.setVisible(true);
    }

    private void clearFields() {
        nameTF.setText("");
        mobileTF.setText("");
        genderGroup.clearSelection();
        deptCB.setSelectedIndex(0);
        joiningTF.setText("");
        salaryTF.setText("");
        roomTF.setText("");
        bmdcTF.setText("");
    }
}
```

STRUCTURAL PATTERN (ADAPTER)

Create DoctorMenuAdapter.java

```java
public class DoctorMenuAdapter {



    private IDoctorService doctorService;



    public DoctorMenuAdapter() {

        doctorService = new DoctorService();

    }



    public void openAddDoctorUI() {

        AddDoctorUI ui = new AddDoctorUI(doctorService);

        ui.setVisible(true);

    }

}
```

Before DrMenu.java

```java
import javax.swing.*;
import java.awt.*;

You, last month | 1 author (You)
public class DrMenu extends JFrame {

    public DrMenu() {
        super(title: "Doctor Menu");
        setSize(width: 400, height: 300);
        setLocationRelativeTo(c: null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JLabel label = new JLabel(text: "This is the Doctor Menu", SwingConstants.CENTER);
        label.setFont(new Font(name: "Arial", Font.BOLD, size: 18));
        add(label);
        JButton addDoctorButton = new JButton(text: "ADD DOCTOR");
        addDoctorButton.setBounds(x: 75, y: 380, width: 200, height: 40); // adjust X, Y to fit
        addDoctorButton.setBackground(Color.BLACK);
        addDoctorButton.setForeground(Color.WHITE);
        addDoctorButton.setFont(new Font(name: "Cascadia Code SemiBold", Font.PLAIN, size: 16))
        addDoctorButton.addActionListener(e -> {
        IDoctorService doctorService = new DoctorService(); // Use the service
        AddDoctorUI addDoctorUI = new AddDoctorUI(doctorService);
        addDoctorUI.setVisible(b: true);        You, last month • SOLID
        this.setVisible(b: false); // Hide menu while AddDoctor form is open
});


    }
}
```

After DrMenu.java

```java
import javax.swing.*;
import java.awt.*;

You, last month | 1 author (You)
public class DrMenu extends JFrame {

    public DrMenu() {
        super(title: "Doctor Menu");
        setSize(width: 400, height: 300);
        setLocationRelativeTo(c: null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JLabel label = new JLabel(text: "This is the Doctor Menu", SwingConstants.CENTER);
        label.setFont(new Font(name: "Arial", Font.BOLD, size: 18));
        add(label);
        JButton addDoctorButton = new JButton(text: "ADD DOCTOR");
        addDoctorButton.setBounds(x: 75, y: 380, width: 200, height: 40); // adjust X, Y to fit
        addDoctorButton.setBackground(Color.BLACK);
        addDoctorButton.setForeground(Color.WHITE);
        addDoctorButton.setFont(new Font(name: "Cascadia Code SemiBold", Font.PLAIN, size: 16))
        DoctorMenuAdapter adapter = new DoctorMenuAdapter();

        addDoctorButton.addActionListener(e -> {
            adapter.openAddDoctorUI();
            this.setVisible(b: false);      You, last month • Uncommitted changes
        });


    }
}
```

Adapter Pattern was used to integrate the legacy DrMenu with the refactored AddDoctorUI module without modifying existing menu logic.

BEHAVIORAL PATTERN(OBSERVER)

The hospital system needs notifications.
This is the BEST behavioral pattern for the system.

Create DoctorObserver.java

```java
public interface DoctorObserver {
    void update(String message);
}
```

Create DoctorNotification.java

```java
import javax.swing.*;

public class DoctorNotification implements DoctorObserver {

    @Override
    public void update(String message) {
        JOptionPane.showMessageDialog(null, message);
    }
}
```

Before DoctorService.java

```java
...

import java.util.ArrayList;
import java.util.List;

...
public class DoctorService implements IDoctorService {
    private List<Doctor> doctors = new ArrayList<>();

    @Override
    public void addDoctor(Doctor doctor) {
        doctors.add(doctor);

    }

    public List<Doctor> getDoctors() {
        return doctors;
    }
}
```

After DoctorService.java

```
import java.util.*;

public class DoctorService implements IDoctorService {

    private List<Doctor> doctors = new ArrayList<>();
    private List<DoctorObserver> observers = new ArrayList<>();

    public void addObserver(DoctorObserver observer) {
        observers.add(observer);
    }

    private void notifyObservers(String message) {
        for (DoctorObserver observer : observers) {
            observer.update(message);
        }
    }

    @Override
    public void addDoctor(Doctor doctor) {
        doctors.add(doctor);
        notifyObservers("Doctor added successfully");
    }
}
```

Observer Pattern was applied to notify UI components whenever a new doctor is added to the system.

The change After adding a behaviour pattern.

AddDoctorUI.java

```java
public class AddDoctorUI extends JFrame {

    private IDoctorService doctorService;

    private JTextField nameTF, mobileTF, joiningTF, salaryTF, roomTF, bmdcTF;
    private JRadioButton maleRB, femaleRB, otherRB;
    private JComboBox<String> deptCB;
    private JButton addButton, backButton;
    private ButtonGroup genderGroup;

    public AddDoctorUI(IDoctorService doctorService) {
        super(title: "Add Doctor Form");
        this.doctorService = doctorService;

        // Behavioral Design Pattern: Observer
        DoctorService service = new DoctorService();
        service.addObserver(new DoctorNotification());        You, 27 seconds ago

        setSize(width: 900, height: 600);
        setLocationRelativeTo(c: null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        initUI();
    }
```

Whole code of **AddDoctorUI.java**

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class AddDoctorUI extends JFrame {

    private IDoctorService doctorService;

    private JTextField nameTF, mobileTF, joiningTF, salaryTF, roomTF, bmdcTF;
    private JRadioButton maleRB, femaleRB, otherRB;
    private JComboBox<String> deptCB;
    private JButton addButton, backButton;
    private ButtonGroup genderGroup;
```

```java
public AddDoctorUI(IDoctorService doctorService) {
    super("Add Doctor Form");
    this.doctorService = doctorService;

    // Behavioral Design Pattern: Observer
    DoctorService service = new DoctorService();
    service.addObserver(new DoctorNotification());

    setSize(900, 600);
    setLocationRelativeTo(null);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    initUI();
}

private void initUI() {
    JPanel panel = new JPanel();
    panel.setLayout(null);
    panel.setBackground(new Color(143,207,225));

    Font labelFont = new Font("Cascadia Code SemiBold", Font.PLAIN, 18);

    JLabel titleLabel = new JLabel("Add Doctor Form");
    titleLabel.setBounds(130, 20, 300, 40);
    titleLabel.setFont(new Font("Biome", Font.BOLD, 30));
    panel.add(titleLabel);

    JLabel nameLabel = new JLabel("Doctor Name:");
    nameLabel.setBounds(75, 80, 120, 25);
    nameLabel.setFont(labelFont);
    panel.add(nameLabel);

    nameTF = new JTextField();
    nameTF.setBounds(200, 80, 150, 25);
    panel.add(nameTF);

    JLabel mobileLabel = new JLabel("Mobile Number:");
    mobileLabel.setBounds(75, 120, 120, 25);
    mobileLabel.setFont(labelFont);
```

```
        panel.add(mobileLabel);

        mobileTF = new JTextField();
        mobileTF.setBounds(200, 120, 150, 25);
        panel.add(mobileTF);

        JLabel genderLabel = new JLabel("Gender:");
        genderLabel.setBounds(75, 160, 100, 25);
        genderLabel.setFont(labelFont);
        panel.add(genderLabel);

        maleRB = new JRadioButton("Male");
        maleRB.setBounds(150, 160, 80, 25);
        maleRB.setBackground(panel.getBackground());

        femaleRB = new JRadioButton("Female");
        femaleRB.setBounds(230, 160, 80, 25);
        femaleRB.setBackground(panel.getBackground());

        otherRB = new JRadioButton("Other");
        otherRB.setBounds(310, 160, 80, 25);
        otherRB.setBackground(panel.getBackground());

        genderGroup = new ButtonGroup();
        genderGroup.add(maleRB);
        genderGroup.add(femaleRB);
        genderGroup.add(otherRB);

        panel.add(maleRB);
        panel.add(femaleRB);
        panel.add(otherRB);

        JLabel deptLabel = new JLabel("Department:");
        deptLabel.setBounds(75, 200, 120, 25);
        deptLabel.setFont(labelFont);
        panel.add(deptLabel);

        String[] departments = {"CARDIOLOGY", "NEUROSURGERY", "ONCOLOGIST",
"ORTHOPEDICS"};
```

```
deptCB = new JComboBox<>(departments);
deptCB.setBounds(200, 200, 150, 25);
panel.add(deptCB);

JLabel joiningLabel = new JLabel("Joining Date:");
joiningLabel.setBounds(75, 240, 120, 25);
joiningLabel.setFont(labelFont);
panel.add(joiningLabel);

joiningTF = new JTextField();
joiningTF.setBounds(200, 240, 150, 25);
panel.add(joiningTF);

JLabel salaryLabel = new JLabel("Salary:");
salaryLabel.setBounds(75, 280, 120, 25);
salaryLabel.setFont(labelFont);
panel.add(salaryLabel);

salaryTF = new JTextField();
salaryTF.setBounds(200, 280, 150, 25);
panel.add(salaryTF);

JLabel roomLabel = new JLabel("Room No:");
roomLabel.setBounds(75, 320, 120, 25);
roomLabel.setFont(labelFont);
panel.add(roomLabel);

roomTF = new JTextField();
roomTF.setBounds(200, 320, 150, 25);
panel.add(roomTF);

JLabel bmdcLabel = new JLabel("BM&DC REG:");
bmdcLabel.setBounds(75, 360, 120, 25);
bmdcLabel.setFont(labelFont);
panel.add(bmdcLabel);

bmdcTF = new JTextField();
bmdcTF.setBounds(200, 360, 150, 25);
panel.add(bmdcTF);
```

```
    addButton = new JButton("ADD");
    addButton.setBounds(500, 470, 100, 40);
    addButton.setBackground(Color.GREEN);
    addButton.setForeground(Color.WHITE);
    addButton.addActionListener(e -> onAdd());
    panel.add(addButton);

    backButton = new JButton("BACK");
    backButton.setBounds(620, 470, 100, 40);
    backButton.setBackground(Color.RED);
    backButton.setForeground(Color.WHITE);
    backButton.addActionListener(e -> onBack());
    panel.add(backButton);

    this.add(panel);
}

private void onAdd() {
    String name = nameTF.getText();
    String mobile = mobileTF.getText();
    String gender = maleRB.isSelected() ? "Male" :
                femaleRB.isSelected() ? "Female" : "Other";
    String department = (String) deptCB.getSelectedItem();
    String joiningDate = joiningTF.getText();
    double salary = Double.parseDouble(salaryTF.getText());
    String room = roomTF.getText();
    String bmdc = bmdcTF.getText();

    DoctorFactory factory = new DoctorFactory();

    Doctor doctor = factory.createDoctor(
        name, mobile, gender, department,
        joiningDate, salary, room, bmdc
    );

    doctorService.addDoctor(doctor);

    JOptionPane.showMessageDialog(this, "Doctor added successfully!");
```

```
        clearFields();
    }

    private void onBack() {
        this.setVisible(false);
        DrMenu dm = new DrMenu();
        dm.setVisible(true);
    }

    private void clearFields() {
        nameTF.setText("");
        mobileTF.setText("");
        genderGroup.clearSelection();
        deptCB.setSelectedIndex(0);
        joiningTF.setText("");
        salaryTF.setText("");
        roomTF.setText("");
        bmdcTF.setText("");
    }
}
```

Creational Pattern (SINGLETON PATTERN + BUILDER PATTERN)

Before: This created multiple objects in memory, leading to data inconsistency and repeated file loading. Patient and Doctor classes used only traditional constructors.

PList()

```
package Classes;

import Interfaces.*;

import java.io.*;

import java.util.Scanner;

public class PList extends AbstractFileRepository<Patient> implements IPList {

    private static final String PATIENT_FILE_PATH = "Files/PatientList.txt";

    private static PList instance;

    private PList() {

        super(PATIENT_FILE_PATH);

    }

    public static synchronized PList getInstance() {

        if (instance == null) {

            instance = new PList();

        }

        return instance;

    }



    @Override
```

```
  protected void loadFromFile() {

      File file = new File(filePath);

      if (!file.exists()) return;

      try (Scanner sc = new Scanner(file)) {

          while (sc.hasNext()) {

              String id = sc.nextLine();

              String name = sc.nextLine();

              String mobileNo = sc.nextLine();

              String gender = sc.nextLine();

              String age = sc.nextLine();

              String address = sc.nextLine();

              String appointedDoctor = sc.nextLine();

              String room = sc.nextLine();

              String diagnosis = sc.nextLine();

              String deposit = sc.nextLine();

              if (sc.hasNextLine()) sc.nextLine();



              Patient p = new Patient(id, name, mobileNo, gender, age, address, appointedDoctor, room, diagnosis,
deposit);

              items.add(p);

          }

      } catch (Exception ignored) {}

  }
```

```java
@Override
protected String buildBlock(Patient p) {
    return p.getId() + "\n" + p.getName() + "\n" + p.getMobileNo() + "\n" +
            p.getGender() + "\n" + p.getAge() + "\n" + p.getAddress() + "\n" +
            p.getAppointedDoctor() + "\n" + p.getRoom() + "\n" +
            p.getDiagnosis() + "\n" + p.getDeposit() + "\n\n";
}
@Override
protected boolean blocksMatch(Patient p, String block) {
    return buildBlock(p).equals(block);
}
@Override
public void addPatient(Patient p) {
    add(p);
}


@Override
public int searchPatient(String id) {
    return searchByKey(id, Patient::getId);
}
@Override
public Patient getPatient(int index) {
```

```
        return get(index);

    }

    @Override

    public void deletePatient(Patient p) {

        delete(p);

    }

    @Override

    public void updatePatient(String patientToUpdate, Patient updatedPatient) {

        update(patientToUpdate, updatedPatient);

    }

}
```

RegDoctorList()

```
package Classes;

import java.io.*;
import java.util.Scanner;

public class RegDoctorList extends AbstractFileRepository<Doctor> {
    private static final String DOCTOR_FILE_PATH = "Files/DoctorList.txt";
    private static RegDoctorList instance;

    // Private constructor - only calls super once
    private RegDoctorList() {
        super(DOCTOR_FILE_PATH);
    }

    public static synchronized RegDoctorList getInstance() {
        if (instance == null) {
            instance = new RegDoctorList();
```

```java
    }
    return instance;
}

@Override
protected void loadFromFile() {
    File file = new File(filePath);
    if (!file.exists()) return;

    try (Scanner sc = new Scanner(file)) {
        while (sc.hasNext()) {
            String id = sc.nextLine();
            String name = sc.nextLine();
            String mobileNo = sc.nextLine();
            String gender = sc.nextLine();
            String department = sc.nextLine();
            String joiningDate = sc.nextLine();
            String bmdcReg = sc.nextLine();
            String password = sc.nextLine();
            if (sc.hasNextLine()) sc.nextLine();

            Doctor d = new Doctor(id, name, mobileNo, gender, department, joiningDate, bmdcReg, password);
            items.add(d);
        }
    } catch (Exception ignored) {}
}

@Override
protected String buildBlock(Doctor d) {
    return d.getId() + "\n" + d.getName() + "\n" + d.getMobileNo() + "\n" + d.getGender() + "\n" +
            d.getDepartment() + "\n" + d.getJoiningDate() + "\n" + d.getBmdcReg() + "\n" +
            d.getPassword() + "\n\n";
}

@Override
protected boolean blocksMatch(Doctor d, String block) {
    return buildBlock(d).equals(block);
}
```

```java
    // Original helper methods
    public int doctorExists(String name) {
        for (int i = 0; i < items.size(); i++) {
            if (items.get(i).getName().equals(name)) {
                return i;
            }
        }
        return -1;
    }

    public Doctor checkPassword(int index, String pass) {
        if (index < 0 || index >= items.size()) return null;
        Doctor d = items.get(index);
        return d.getPassword().equals(pass) ? d : null;
    }

    public Doctor getDoctor(int index) {
        return get(index);
    }

    public String[] getDoctorNames() {
        String[] names = new String[items.size()];
        for (int i = 0; i < items.size(); i++) {
            names[i] = "Dr. " + items.get(i).getName();
        }
        return names;
    }

    public void addDoctor(Doctor d) {
        add(d);
    }
}
```

Structural Pattern (FACADE PATTERN)

Before: GUI classes directly accessed multiple repositories **(PList, RegDoctorList, AppointmentList, PrescriptionList)**. This resulted in tight coupling between the UI layer and the data layer.

After: We introduced **HospitalFacade.java** as a single unified interface for all hospital operations. All list managers are now accessed through this facade.

New File: HospitalFacade.java

```
package Classes;

public class HospitalFacade {
    private final PList patientManager = PList.getInstance();
    private final RegDoctorList doctorManager = RegDoctorList.getInstance();
    private final AppointmentList appointmentManager = AppointmentList.getInstance();
    private final PrescriptionList prescriptionManager = PrescriptionList.getInstance();

    // Patient operations
    public void addPatient(Patient p) {
        patientManager.addPatient(p);
    }

    public Patient findPatient(String id) {
        int idx = patientManager.searchPatient(id);
        return patientManager.getPatient(idx);
    }

    public void deletePatient(Patient p) {
        patientManager.deletePatient(p);
    }

    // Doctor operations
    public void addDoctor(Doctor d) {
        doctorManager.addDoctor(d);
    }

    public String[] getDoctorNames() {
        return doctorManager.getDoctorNames();
```

```
    }

    public Doctor findDoctorByName(String name) {
        int idx = doctorManager.doctorExists(name);
        return (idx != -1) ? doctorManager.getDoctor(idx) : null;
    }

    // Appointment operations
    public void bookAppointment(Appointment app) {
        appointmentManager.addAppointment(app);
    }

    public Appointment findAppointment(String appId) {
        int idx = appointmentManager.searchAppointment(appId);
        return appointmentManager.getAppointment(idx);
    }

    public void deleteAppointment(Appointment app) {
        appointmentManager.deleteAppointment(app);
    }

    // Prescription operations
    public void issuePrescription(Prescription p) {
        prescriptionManager.addPrescription(p);
    }

    public Prescription findPrescription(String patientId) {
        int idx = prescriptionManager.searchPrescription(patientId);
        return prescriptionManager.getPrescription(idx);
    }
}
```

Recommendation: All GUI classes should now use HospitalFacade instead of calling repositories directly (partial integration done; full integration recommended in next iteration).

Benefit: Reduced coupling and simplified interaction with the subsystem.

Behavioral Pattern (TEMPLATE METHOD PATTERN)

Before: Significant code duplication existed in file I/O operations (loading, building blocks, deleting, and updating) across PList.java, AppointmentList.java, and RegDoctorList.java. Each class repeated similar logic for reading/writing files.

After: We created **AbstractFileRepository.java** and applied the Template Method Pattern. Common file operations are defined in the abstract parent class, while subclasses only implement specific steps.

New File: AbstractFileRepository.java

```
import java.io.*;
import java.util.ArrayList;
import java.util.List;

public abstract class AbstractFileRepository<T> {
    protected final String filePath;
    protected final List<T> items = new ArrayList<>();

    protected AbstractFileRepository(String filePath) {
        this.filePath = filePath;
    }

    protected abstract void loadFromFile();
    protected abstract String buildBlock(T item);
    protected abstract boolean blocksMatch(T item, String block);

    public void add(T item) {
        items.add(item);
        appendToFile(item);
    }

    private void appendToFile(T item) {
        try (FileWriter fw = new FileWriter(filePath, true)) {
            fw.write(buildBlock(item));
        } catch (IOException ex) {
```

```
        throw new RuntimeException("Error saving to file", ex);
    }
}

public void delete(T item) {
    items.remove(item);
    rewriteFileExcluding(buildBlock(item));
}

public void update(String oldBlock, T updatedItem) {
    rewriteFileReplacing(oldBlock, buildBlock(updatedItem));
}

private void rewriteFileExcluding(String blockToRemove) {
    rewriteFile(blockToRemove, null);
}

private void rewriteFileReplacing(String oldBlock, String newBlock) {
    rewriteFile(oldBlock, newBlock);
}

private void rewriteFile(String targetBlock, String replacement) {
    try {
        List<String> lines = readAllLines();
        StringBuilder newContent = new StringBuilder();
        StringBuilder current = new StringBuilder();

        for (String line : lines) {
            current.append(line).append("\n");
            if (current.toString().endsWith("\n\n")) {
                String block = current.toString();
                if (block.equals(targetBlock)) {
                    if (replacement != null) newContent.append(replacement);
                } else {
                    newContent.append(block);
                }
                current = new StringBuilder();
            }
        }
```

```java
        try (FileWriter fw = new FileWriter(filePath)) {
            fw.write(newContent.toString());
        }
    } catch (IOException ex) {
        throw new RuntimeException("Error rewriting file", ex);
    }
}

protected List<String> readAllLines() throws IOException {
    List<String> lines = new ArrayList<>();
    try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
        String line;
        while ((line = br.readLine()) != null) {
            lines.add(line);
        }
    }
    return lines;
}

public int searchByKey(String key, java.util.function.Function<T, String> keyExtractor) {
    for (int i = 0; i < items.size(); i++) {
        if (keyExtractor.apply(items.get(i)).equals(key)) {
            return i;
        }
    }
    return -1;
}

public T get(int index) {
    if (index < 0 || index >= items.size()) return null;
    return items.get(index);
}

// NEW PUBLIC METHOD - This fixes the "items has protected access" error
public int getAllItemsSize() {
    return items.size();
}
}
```

PList.java (After):

```java
package Classes;

import Interfaces.*;
import java.io.*;
import java.util.Scanner;

public class PList extends AbstractFileRepository<Patient> implements IPList {
    private static final String PATIENT_FILE_PATH = "Files/PatientList.txt";
    private static PList instance;

    private PList() {
        super(PATIENT_FILE_PATH);
    }

    public static synchronized PList getInstance() {
        if (instance == null) {
            instance = new PList();
        }
        return instance;
    }

    @Override
    protected void loadFromFile() {
        File file = new File(filePath);
        if (!file.exists()) return;

        try (Scanner sc = new Scanner(file)) {
            while (sc.hasNext()) {
                String id = sc.nextLine();
                String name = sc.nextLine();
                String mobileNo = sc.nextLine();
                String gender = sc.nextLine();
```

```
            String age = sc.nextLine();
            String address = sc.nextLine();
            String appointedDoctor = sc.nextLine();
            String room = sc.nextLine();
            String diagnosis = sc.nextLine();
            String deposit = sc.nextLine();
            if (sc.hasNextLine()) sc.nextLine();

            Patient p = new Patient(id, name, mobileNo, gender, age, address, appointedDoctor, room, diagnosis,
deposit);
            items.add(p);
        }
    } catch (Exception ignored) {}
  }

  @Override
  protected String buildBlock(Patient p) {
    return p.getId() + "\n" + p.getName() + "\n" + p.getMobileNo() + "\n" +
          p.getGender() + "\n" + p.getAge() + "\n" + p.getAddress() + "\n" +
          p.getAppointedDoctor() + "\n" + p.getRoom() + "\n" +
          p.getDiagnosis() + "\n" + p.getDeposit() + "\n\n";
  }

  @Override
  protected boolean blocksMatch(Patient p, String block) {
    return buildBlock(p).equals(block);
  }

  @Override
  public void addPatient(Patient p) {
    add(p);
  }

  @Override
  public int searchPatient(String id) {
    return searchByKey(id, Patient::getId);
  }

  @Override
```

```java
    public Patient getPatient(int index) {
        return get(index);
    }

    @Override
    public void deletePatient(Patient p) {
        delete(p);
    }

    @Override
    public void updatePatient(String patientToUpdate, Patient updatedPatient) {
        update(patientToUpdate, updatedPatient);
    }
}
```

Appointment.java

Old Code:

```
package Classes;

public class Appointment {
    private final String appID;
    private final String patientName;
    private final String diagnosis;
    private final String doctorName;
    private final String appDate;
    private final String appTime;

    public Appointment(String appID, String patientName, String diagnosis, String doctorName,
String appDate, String appTime) {
        this.appID = appID;
        this.patientName = patientName;
        this.diagnosis = diagnosis;
        this.doctorName = doctorName;
        this.appDate = appDate;
        this.appTime = appTime;
    }

    public String getAppId() {
        return appID;
    }

    public String getPatientName() {
        return patientName;
    }

    public String getDiagnosis() {
        return diagnosis;
    }

    public String getDoctorName() {
        return doctorName;
    }

    public String getAppDate() {
        return appDate;
    }
```

```
    public String getAppTime() {
        return appTime;
    }
}
```

Issues Identified

1. No input validation: null or empty strings can silently corrupt data
2. Primitive Obsession: appDate and appTime as raw Strings have no format guarantee
3. Construction complexity: 6 parameters in a constructor is error-prone (easy to swap arguments)
4. No **toString()**: makes debugging/logging unnecessarily hard

Refactoring Applied

Pattern used: Builder Pattern Reason: The constructor has 6 parameters of the same type (String). This is a classic use case — callers can easily pass arguments in the wrong order with no compile-time error. The Builder Pattern solves this with a readable, safe construction API.

Refactored Code:

```
package Classes;

import java.time.LocalDate;
import java.time.LocalTime;
import java.time.format.DateTimeParseException;
import java.util.Objects;

public class Appointment {

    private final String appID;
    private final String patientName;
    private final String diagnosis;
    private final String doctorName;
    private final LocalDate appDate;
    private final LocalTime appTime;

    // Private constructor — only Builder can instantiate
    private Appointment(Builder builder) {
        this.appID       = builder.appID;
        this.patientName = builder.patientName;
        this.diagnosis   = builder.diagnosis;
        this.doctorName  = builder.doctorName;
```

```
    this.appDate     = builder.appDate;
    this.appTime     = builder.appTime;
}

// ── Getters ──────────────────────────────────────────────────────────────
public String getAppId()       { return appID; }
public String getPatientName()  { return patientName; }
public String getDiagnosis()    { return diagnosis; }
public String getDoctorName()   { return doctorName; }
public LocalDate getAppDate()   { return appDate; }
public LocalTime getAppTime()   { return appTime; }

@Override
public String toString() {
    return String.format(
        "Appointment{id='%s', patient='%s', doctor='%s', diagnosis='%s', date=%s, time=%s}",
        appID, patientName, doctorName, diagnosis, appDate, appTime
    );
}

// ── Builder ──────────────────────────────────────────────────────────────
public static class Builder {

    // Required fields
    private final String appID;
    private final String patientName;

    // Optional / secondary fields
    private String diagnosis = "Not specified";
    private String doctorName;
    private LocalDate appDate;
    private LocalTime appTime;

    public Builder(String appID, String patientName) {
        this.appID       = requireNonBlank(appID,       "Appointment ID");
        this.patientName = requireNonBlank(patientName, "Patient name");
    }

    public Builder diagnosis(String diagnosis) {
        this.diagnosis = requireNonBlank(diagnosis, "Diagnosis");
        return this;
    }

    public Builder doctorName(String doctorName) {
```

```
    this.doctorName = requireNonBlank(doctorName, "Doctor name");
    return this;
}

/**
 * Accepts ISO-8601 format: yyyy-MM-dd (e.g. "2025-04-07")
 */
public Builder appDate(String date) {
    try {
        this.appDate = LocalDate.parse(date);
    } catch (DateTimeParseException e) {
        throw new IllegalArgumentException(
            "Invalid date format. Expected yyyy-MM-dd, got: " + date
        );
    }
    return this;
}

/**
 * Accepts ISO-8601 format: HH:mm (e.g. "09:30")
 */
public Builder appTime(String time) {
    try {
        this.appTime = LocalTime.parse(time);
    } catch (DateTimeParseException e) {
        throw new IllegalArgumentException(
            "Invalid time format. Expected HH:mm, got: " + time
        );
    }
    return this;
}

public Appointment build() {
    Objects.requireNonNull(doctorName, "Doctor name must be set before building.");
    Objects.requireNonNull(appDate,    "Appointment date must be set before building.");
    Objects.requireNonNull(appTime,    "Appointment time must be set before building.");
    return new Appointment(this);
}

// ── Validation helper ──────────────────────────────────────────────
private static String requireNonBlank(String value, String fieldName) {
    if (value == null || value.isBlank()) {
        throw new IllegalArgumentException(fieldName + " must not be null or blank.");
    }
```

```
            return value;
        }
    }
}
```

AppointmentList.java

Old Code:

```
package Classes;

import java.io.*;
import java.util.Scanner;

public class AppointmentList extends AbstractFileRepository<Appointment> {
    private static final String APPOINTMENT_FILE_PATH = "Files/AppointmentList.txt";
    private static AppointmentList instance;

    private AppointmentList() {
        super(APPOINTMENT_FILE_PATH);
    }

    public static synchronized AppointmentList getInstance() {
    if (instance == null) {
        instance = new AppointmentList();
    }
    return instance;
}

    @Override
    protected void loadFromFile() {
        File file = new File(filePath);
        if (!file.exists()) return;

        try (Scanner sc = new Scanner(file)) {
            while (sc.hasNext()) {
                String appID = sc.nextLine();
                String patientName = sc.nextLine();
                String diagnosis = sc.nextLine();
                String doctorName = sc.nextLine();
                String appDate = sc.nextLine();
                String appTime = sc.nextLine();
```

```
            if (sc.hasNextLine()) sc.nextLine();

            Appointment app = new Appointment(appID, patientName, diagnosis, doctorName,
appDate, appTime);
            items.add(app);
        }
    } catch (Exception ignored) {}
  }

  @Override
  protected String buildBlock(Appointment app) {
    return app.getAppId() + "\n" +
          app.getPatientName() + "\n" +
          app.getDiagnosis() + "\n" +
          app.getDoctorName() + "\n" +
          app.getAppDate() + "\n" +
          app.getAppTime() + "\n\n";
  }

  @Override
  protected boolean blocksMatch(Appointment app, String block) {
    return buildBlock(app).equals(block);
  }

  public void addAppointment(Appointment app) {
    add(app);
  }

  public int searchAppointment(String appId) {
    return searchByKey(appId, Appointment::getAppId);
  }

  public Appointment getAppointment(int index) {
    return get(index);
  }

  public void deleteAppointment(Appointment app) {
    delete(app);
  }

  public void updateAppointment(String appointmentToUpdate, Appointment
updatedAppointment) {
    update(appointmentToUpdate, updatedAppointment);
  }
```

```
}
```

Issues Identified

1. Silent exception swallowing: catch (Exception ignored) {} is the most dangerous line here. A corrupted file, missing line, or I/O error will fail completely silently, leaving items partially loaded with no way to diagnose it.

2. Fragile file parsing: The loadFromFile() reads lines positionally with no validation. If any record is malformed (missing a line, extra blank line), every subsequent record loads with shifted/wrong data. This will corrupt silently due to issue #1.

3. Appointment construction bypasses the Builder: Now that Appointment uses a Builder with validation, constructing it directly with 6 raw strings here breaks that contract entirely.

4. blocksMatch is inefficient: It rebuilds the full block string just to do equality comparison. The intent is just to check if a block belongs to a given appointment, matching on the unique ID is the correct and cheaper approach.

Refactored Code:

```
package Classes;

import java.io.*;
import java.util.Scanner;
import java.util.logging.Level;
import java.util.logging.Logger;

public class AppointmentList extends AbstractFileRepository<Appointment> {

    private static final String APPOINTMENT_FILE_PATH = "Files/AppointmentList.txt";
    private static final Logger LOGGER = Logger.getLogger(AppointmentList.class.getName());

    private static volatile AppointmentList instance;

    private AppointmentList() {
        super(APPOINTMENT_FILE_PATH);
    }

    // ── Singleton
    // volatile + double-checked locking: avoids lock acquisition cost on every
    // call once the instance is initialized, while remaining thread-safe.
```

```java
public static AppointmentList getInstance() {
    if (instance == null) {
        synchronized (AppointmentList.class) {
            if (instance == null) {
                instance = new AppointmentList();
            }
        }
    }
    return instance;
}

// ── File Loading
@Override
protected void loadFromFile() {
    File file = new File(filePath);
    if (!file.exists()) return;

    try (Scanner sc = new Scanner(file)) {
        while (sc.hasNextLine()) {
            Appointment app = parseNextAppointment(sc);
            if (app != null) {
                items.add(app);
            }
        }
    } catch (IOException e) {
        // Catch specific exception, not all of Exception.
        // Fail loudly — silent swallowing hides data corruption.
        LOGGER.log(Level.SEVERE, "Failed to load appointments from file: " + filePath, e);
    }
}

/**
 * Reads exactly 6 data lines + 1 blank separator from the scanner.
 * Returns null and logs a warning if the block is malformed,
 * so one bad record doesn't corrupt all subsequent records.
 */
private Appointment parseNextAppointment(Scanner sc) {
    try {
        String appID       = readLine(sc, "appID");
        String patientName = readLine(sc, "patientName");
        String diagnosis   = readLine(sc, "diagnosis");
        String doctorName  = readLine(sc, "doctorName");
        String appDate     = readLine(sc, "appDate");
        String appTime     = readLine(sc, "appTime");
```

```
        if (sc.hasNextLine()) sc.nextLine(); // consume blank separator

        // Uses the Builder — validation is enforced at construction
        return new Appointment.Builder(appID, patientName)
                .diagnosis(diagnosis)
                .doctorName(doctorName)
                .appDate(appDate)
                .appTime(appTime)
                .build();

    } catch (IllegalArgumentException | IllegalStateException e) {
        // One malformed record is skipped, not the entire file
        LOGGER.log(Level.WARNING, "Skipping malformed appointment record: " +
e.getMessage());
        return null;
    }
}

/**
 * Reads the next line, throwing clearly if the file ends unexpectedly.
 * Replaces silent positional reads that corrupt on missing lines.
 */
private String readLine(Scanner sc, String fieldName) {
    if (!sc.hasNextLine()) {
        throw new IllegalStateException(
            "Unexpected end of file while reading field: " + fieldName
        );
    }
    return sc.nextLine().trim();
}

// —— Serialization
```

```
@Override
protected String buildBlock(Appointment app) {
    return app.getAppId()       + "\n" +
           app.getPatientName() + "\n" +
           app.getDiagnosis()   + "\n" +
           app.getDoctorName()  + "\n" +
           app.getAppDate()     + "\n" +
           app.getAppTime()     + "\n\n";
}
```

```
    // ── Match fix: ID-based instead of full block rebuild ──────────────────────────
    // Old: rebuilt the entire block string just for equality — fragile and wasteful.
    // New: matches on the unique appID — correct by design, O(1) string compare.
    @Override
    protected boolean blocksMatch(Appointment app, String block) {
        return block.startsWith(app.getAppId());
    }

    // ── Public API
──────────────────────────────────────────────────────────────────────────────────
    public void addAppointment(Appointment app)                    { add(app); }
    public Appointment getAppointment(int index)                   { return get(index); }
    public void deleteAppointment(Appointment app)                 { delete(app); }
    public int searchAppointment(String appId)                     { return searchByKey(appId,
Appointment::getAppId); }
    public void updateAppointment(String appId, Appointment updatedApp)      { update(appId,
updatedApp); }
}
```